

Building the Decentralized Future: Blockchain Innovation with Ethereum at the Core

Prince Solanki¹, Sanjay Kumar², Amogh Sharma³, Dhruv Mehta⁴, Goutam Kumar⁵, and Vinay Shukla^{6*}

¹ Department of Mathematics, School of Science, IILM University, Greater Noida, Uttar Pradesh, India

prince@iilm.edu

² Department of Mathematics, School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh, India

sanjayyadav2276@gmail.com

³ School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh, India

S24CSEU1212@bennett.edu.in

⁴ School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh, India

S24CSEU0816@bennett.edu.in

⁵ School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh, India

S24CSEU1214@bennett.edu.in

⁶ *Department of Mathematics, School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh, India

vsshukla09@gmail.com

*Corresponding author

Abstract. Web 3.0, the next iteration of the internet, promises a decentralized future where users have control over their data and digital assets. At the heart of this new movement is blockchain technology—a technology that provides transparency, immutability, and user control. Ethereum, specifically, has positioned itself as a platform for constructing decentralized applications (dApps) driven by smart contracts. Through the provision of programmable, self-executing code free from central intermediaries, Ethereum redefines the potential for digital services to function. But beyond this capability is a cost model that quietly but profoundly influences what can be afforded to build by developers. The findings suggest that inserting new data costs the most, while delete operations were rewarded with partial gas refunds—an incentive system that resonates with Ethereum’s emphasis on lean, optimized state usage.

Keywords: Blockchain · Ethereum · Decentralization · Smart Contracts · dApp · Web 3.0 · Gas Cost.

1 Introduction

Web 3.0 is supposed to be the third generation of the internet and is centered around the themes of decentralization, user ownership, and trustless interactions.

Basically, blockchain technology introduces secure, transparent ways of handling data; the development of dApps and smart contracts, which was pioneered by Ethereum brought a new paradigm into being: the authority and control over the centralized entity ends there and comes to the user. This paper explores the symphony of Web 3, blockchain, and Ethereum as a way of predicting the future of the Internet [2].

Web 1.0 refers to the earliest version of the World Wide Web that existed up until around the early 2000s. It was a static (fixed) web where users could only view and read content, but could not interact with or contribute to the websites. Technology: Websites were mainly built using technologies like HTML, HTTP, URI, CSS, and XML. No user-generated content: Only a few people, who knew how to design web pages, could create or modify websites. Slow and limited: Users had to manually refresh web pages to see new content, and the internet speed was often slow. In Web 1.0, information was displayed as raw data, and users could search for information, but they couldn't make changes [10].

Growth of Web 1.0 (1990s): The first site, info.cern.ch, came into service, explaining how the web worked in 1993. This actually made web pages accessible, including text and images, more interactive. Netscape Navigator was released in 1994, one of the first really widely used web browsers, it helped in making the World Wide Web popular [2].

Decline and Transition to Web 2.0: The term Web 2.0 was first coined by Dale Dougherty and Tim O'Reilly in the early 2000s. In fact, it is known as the "read and write" web, which enables two-way communication. Rather than a model for reading only, like Web 1.0, users can upload, share, or create content. Fundamental technologies thus are RSS, Atom, RDF, AJAX, and JavaScript, which allow dynamic, interactive websites and user-created material. Thus, this new wave was accompanied by the creation of social media, blogs, and common cooperation spaces like Wikipedia. While Web 2.0 has promoted user interaction, there is a growing concern about privacy and security issues resulting from access to personal data and vulnerability to hacking. Websites, for example, Wikipedia, Amazon, and eBay, were among the most important vehicles in this shift toward interactive and community-based experiences. Platforms like Facebook in 2004 and YouTube in 2005 shifted the Web usage orientation from being a read-only environment to one where one can create and share content easily [6].

Through Web 3.0, the interaction of Internet surfers with the Internet is revolutionized, dynamic applications are provided with fluid interaction in 3D graphics and virtual environments, and it is not keyword-driven as the web is presently. Instead, semantic understanding makes machines process and comprehend better, and the web thus becomes more intelligent and per personal as it becomes context-aware, thereby customizing experiences in particular for each user. This, however, remains an evolving web with much promise and shows many challenges on its face. These include managing user identities securely, making the complexity of client-server interaction greater, and the potential to create intelligent applications that could fulfill very high expectations. Once these challenges have been overcome, Web 3.0 will hold tremendous promise to

offer a highly immersive, customizable, decentralized experience [4] that may reshape our digital interactions from top to bottom.

Motivation: This essay ventures into Ethereum’s gas cost and storage architecture—not as an afterthought, but as the driving force behind dApp design. It asks a directive question: are programmers merely shelling out for storage space, or are they navigating an integrated economic system that influences them toward certain choices, habits, and compromises?

Main contributions: To fix this, we created and deployed a proof-of-concept voting dApp that simulates typical contract operations: inserting, updating, and deleting persistent state. Every function was thoroughly tested to assess its gas consumption and consequent effect on storage state. The research exceeds the scope of high-level trends and examines the fine-grained cost behavior influencing the day-to-day reality of developing on Ethereum.

The rest of this paper is structured as follows: Section 2 reviews pertinent work and situates Ethereum’s place in Web 3.0’s development. Section 3 outlines the experimental approach and deployment framework. Section 4 presents the results and their smart contract design implications. Section 5 benchmarks present study with the existing ones. Section 6 describes the technical configuration. Section 7 discusses the limitations, and Section 8 concludes with insights and directions for future research in optimizing dApp building for Ethereum’s future ecosystem.

2 Literature review

Across different sectors, especially finance, governance, and digital asset management, India has rapidly emerged as a leader in adopting Web 3 technologies. Decentralized technologies like blockchain unlock the innovative power, increase transparency, security, and efficiency. Indian businesses and individuals are transforming their relationship with digital assets because of the DeFi and digital identity systems under way in Web 3. Government and Institutional Support: The Indian government is investing some crucial resources, and efforts are being made in order to unlock the opportunities associated with blockchain technology that have been opened up. A few examples of these include Kerala Blockchain Academy, through which it integrates blockchains into public service enhancement programs [6]. The Government of India, via the Ministry of Electronics and Information Technology (MeitY), initiated Vishvasya-Blockchain Technology Stack that promotes Blockchain-as-a-Service (BaaS) [5].

One of the greatest breakthroughs in the history of decentralized technology is the creation of Ethereum, as suggested by Vitalik Buterin in 2014. In his whitepaper [2], Buterin presented a programmable blockchain that advanced the use of Bitcoin beyond that of cryptocurrency transactions by allowing the deployment of smart contracts using a Turing-complete virtual machine called the Ethereum Virtual Machine (EVM). In contrast to Bitcoin’s restricted scripting functionality, Ethereum facilitates the deployment of decentralized applications

(dApps) that can execute sophisticated operations independently on-chain, making new governance, identity, and finance paradigms possible.

Yet another milestone turning point in the development of Ethereum is the creation of Casper, Ethereum's Proof-of-Stake (PoS) protocol by Vlad Zamfir in 2015 in his paper [12]. Casper was created to correct the energy wastefulness and centralization issues surrounding Proof-of-Work (PoW), Ethereum's initial consensus algorithm. Zamfir's Casper brought about a combined PoW/PoS scheme in which validators supplant miners and are chosen on the basis of how much Ether they "stake" as collateral. The validators propose and attest to new blocks, with slashing conditions being levied against dishonesty or extended idleness.

A key technical document for the Ethereum ecosystem is Gavin Wood's 2014 Yellow Paper [10]. Unlike the more conceptual overview by Buterin, Wood's Yellow Paper serves as the formal technical specification of the Ethereum protocol. It describes Ethereum as a transaction-based state machine that allows the execution of smart contracts through a globally distributed network of nodes. The paper introduces the Ethereum Virtual Machine (EVM) as an environment for executing smart contract code reliably, ensuring consistent outputs across all validating nodes in the network.

Indian Web 3 startup ecosystem is booming with innovative companies like the one that launched in digital asset security-Cypherock. Here, research on multiple Web 3 applications like DeFi and enterprise tech is being pursued by startups, according to the latest FORBES INDIA. Most are excited about Web 3, but ambiguity over regulations is one of the key problems here [8].

Blockchain, a revolutionary digital technology, is a decentralized and immutable ledger that records transactions and changes across a varied network of nodes. Starting with Bitcoin in 2008, blockchain operates through linking blocks in a chronological manner, ensuring clarity and security. Key use cases of blockchain are as follows [13]-

1. Decentralisation
2. Removing intermediaries [12]
3. Tamper-proof transactions
4. Auditability (transparency, traceability, and authenticity)

All of these are collectively used under the fields of cryptocurrency, smart contracts, and IoT. Blockchain caters to data authenticity challenges but there is issues caused like scalability and energy consumption overload [7].

The Ethereum system is an open-source blockchain project founded by Vitalik Buterin [2] on July 30, 2015, that can develop a decentralised application using the Turing-complete Ethereum Virtual Machine [10] and supports smart contracts of which the programming language is called Solidity [10] to code in customized rules as well as even a custom transaction format. It serves of like a kind of transaction-based state machine and is powered by native cryptocurrency, Ether, and a gas fee system [11] for transaction costs. It has two types of accounts: Externally Owned Accounts that represent user-to-user interactions and Contract Accounts for executing smart contracts.

The work on the Ethereum network is distributed over a de-centralized network: they execute smart contracts and process transactions. The blockchain changed from an energy-intensive variant called Proof of Work (PoW) to a more energy-friendly Proof of Stake (PoS)[12] mechanism, thereby saving immense amounts of energy. Ethereum transactions have a resource-intense measuring unit, called gas units, for which users pay in Ether. High network activity leads to congestion, thus a high gas fee. Despite its scalability problems, Ethereum’s Layer 2 solutions strive to improve throughput while relieving some workload off the main network.

This study infers that the efficiency and cost of data storage within Ethereum smart contracts are crucial determinants of decentralized application (dApps) design and scalability within the Web 3 environment. It suggests that Ethereum’s storage paradigm—characterized by intense gas fees [9] associated with permanent storage and rewards for erasure—tends to mold application design directly for developers, trading off functionality, expense, and decentralization.

At the center of Ethereum’s architecture is a decentralized, tamper-proof ledger in which all smart contract information is redundantly stored on thousands of nodes. This offers high security and trustworthiness but results in heavy resource usage. Consequently, gas costs for writing to permanent storage are much greater than for computation or temporary memory operations. The hypothesis posits that this cost of storage is not just a technical limitation but an important economic factor that determines how developers design their smart contracts. To explore this, the research considers three typical storage operations in dApps:

1. Inserting new data (e.g., adding a user or candidate), which uses the most gas because of the creation of new storage slots and network-wide consensus to make them persistent.
2. Updating already present data (e.g., incrementing a counter), which is relatively costly as it changes already present entries in Ethereum’s state.
3. Data deletion (clearing a storage slot to zero), which is the cheapest and usually leads to a gas refund, motivating developers to delete old or unused data.

On the basis of these operations, the hypothesis also predicts that Ethereum’s expensive storage encourages developers to adopt optimized and minimalist smart contract designs. The cost management strategies are:

1. Minimizing on-chain storage, rather than utilizing off-chain databases or decentralized storage networks (e.g., IPFS, Arweave) [3] and keeping only critical data or references on-chain.
2. Utilizing gas-efficient design patterns, e.g., data packing and compact structure usage, to minimize storage slot usage.
3. Utilizing Layer 2 scaling solutions (e.g., Optimism, zk-Rollups) to move operations off the Ethereum main-net and reduce fees.
4. Utilizing Ethereum’s gas refund mechanism to recover expenses by removing unnecessary state variables.

3 Methodology and Implementation

This research examines the cost dynamics of data operations within Ethereum smart contracts and their design implications for decentralized application (dApp) design. A quantitative, experiment-based approach was used to analyze how persistent storage influences gas usage and contract scalability on Ethereum.

1. **Experimental Design:** A prototype smart contract for voting was created with Solidity intended to mimic the typical dApp situation involving permanent storage. Three main operations, each for a different storage operation, were performed in the contract:
 - Insertion of a new candidate, as the insertion of a new storage slot (write operation).
 - Vote casting, updating the vote total of an already present candidate (update operation).
 - Removal of a candidate, as resetting the value of a storage slot to zero (delete operation).
2. **Deployment Environment:** The contract was deployed onto the Ethereum Sepolia testnet via the Hardhat development environment. Interactions were scripted out using web3.js [7], allowing reproducible and consistent calling of contract functions. Gas used data for every transaction was monitored through the Sepolia testnet’s public blockchain explorer and the Web3 API.
3. **Data Collection:** In order to guarantee statistical accuracy, every operation type was performed 50 times under uniform conditions. The following data were captured:
 - Cost of gas per transaction for each operation type.
 - Storage modifications before and after each transaction, ensuring whether new information was added, old information was updated, or information was actually deleted.
4. **Analysis Tools and Workflow:** Gas cost logs were exported in CSV format and analyzed using Python, with the pandas and matplotlib libraries for data analysis and plotting. Ethereum’s JSON-RPC API was employed to check storage state changes at the contract level. This two-layer analysis ensured both gas and storage metrics were correctly interpreted. Detailed workflow of the algorithm is represented by Figure 1.
5. **Evaluation Metrics:** The experiment tested the following metrics:
 - Average gas cost per operation type.
 - Variance in gas fees to observe network anomalies or edge conditions.
 - Refund of net gas for data removal.
 - Storage usage over time, reflecting data persistence and removal.
6. **Visualization:** The experiment tested the following metrics:
 - Comparison of average gas fees for add, update, and remove operations was represented via a bar graph.
 - The increase and decrease in contract storage with time, as new candidates were added and removed, were graphed via a line graph.

7. **Assumptions and Controls:** To ensure storage-related gas expenses are isolated:
- All transactions were performed under comparable network circumstances to minimize gas price volatility.
 - Smart contract logic was deliberately kept simple in order to minimize non-storage-based computation.
 - Temporary memory and stack manipulations were not measured since they do not involve persistent gas expenses.

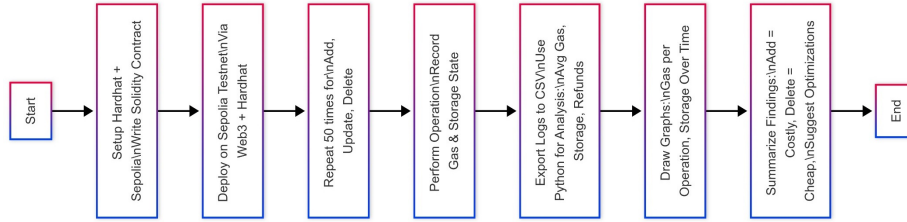


Fig. 1. Algorithm workflow.

4 Results and discussion

Ethereum, being a decentralized platform, has experienced explosive growth in Web 3 applications, or decentralized applications (dApps). Nevertheless, as Ethereum usage grows, it is essential to comprehend the implication of Ethereum’s storage costs in developing scalable and cost-effective dApps.

The experiment identified obvious differences in gas expenditure and storage tendencies across different smart contract operations on Ethereum (see Figure 2 and Figure 2):

1. Data Adding (such as adding a new candidate to the register) was the most gas-intensive operation on average, coming at about 45,000 gas per transaction. This excessive expenditure is mostly a result of establishing new permanent storage slots within the blockchain.
2. Updating Old Data (voting) took a moderate average gas cost of about 25,000 gas. Because this operation is typically an overwrite of previously allocated storage, it uses less gas than when initializing new data.
3. Removing Data (clearing a candidate’s record) costs the least, averaging 8,000 gas. This operation has partial gas refunds, an incentive system that rewards developers for reclaiming unused storage.

In storage usage, the cumulative storage size of the contract grew with the addition of new candidates and reduced when candidates were removed. This

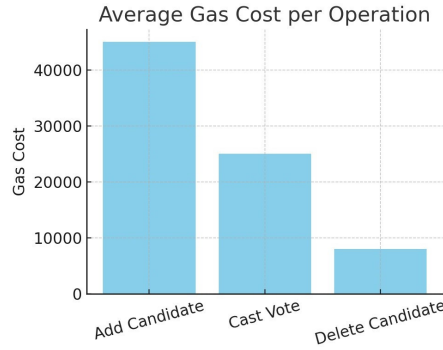


Fig. 2. Average Gas Cost per Operation (in gas units).



Fig. 3. Contract Storage Size Over Time (in storage slots).

Table 1. Gas Cost Comparison for Storage Operations

Operation	Average Gas Cost
Add Candidate	45,000
Cast Vote	25,000
Delete Candidate	8,000

behavior reflects Ethereum’s mutable but persistent storage model. The graphs and Table 1 above provide a clear illustration of the striking difference in gas expenses between operations and the evolving nature of storage usage over time. These observations validate the assumption that storage-based operations are key drivers of gas usage in Ethereum smart contracts. This has significant implications for dApp scalability and architecture, underlining the significance of effective storage management.

5 Comparison with Prior Work

Buterin [2] described the initial vision for Ethereum, a general-purpose blockchain capable of supporting decentralized applications and smart contracts. Although that paper presented the theoretical and architectural foundations of Ethereum’s Turing-complete virtual machine, our work extends these concepts by empirically examining the effects of storage costs on smart contract design choices—an area not thoroughly explored in prior research.

Buterin [1] distinguished between public and private blockchains, focusing primarily on governance and access control. In contrast, our research delves into the economic implications of Ethereum’s public architecture, specifically highlighting gas cost optimization mechanisms that are crucial for enhancing scalability on the public network.

The Ethereum Yellow Paper by Wood [10] offers a comprehensive technical specification of Ethereum, explaining how gas functions and state transitions are

performed. While Wood establishes the rules at the protocol level, our study expands upon this by demonstrating—through controlled experiments—how these rules translate into practical development constraints and trade-offs when implementing dApps, especially those with extensive storage requirements.

Zamfir’s [12] proposal of the Casper protocol laid the foundation for Ethereum’s transition from Proof of Work (PoW) to Proof of Stake (PoS), improving the platform’s energy efficiency and consensus mechanisms. While Casper contributes to Ethereum’s evolution, it primarily addresses consensus-level challenges. Our work, however, focuses on the microeconomics of gas consumption within smart contracts, shedding light on how developers manage efficiency constraints unrelated to consensus but central to contract performance.

6 System Setup

1. **Hardware:** All experiments were conducted on a Lenovo LOQ 15.6" laptop featuring a 13th Gen Intel Core i5 processor, 16 GB RAM, an NVIDIA GeForce RTX 3050 GPU, and a 512 GB SSD. This configuration ensured sufficient computational resources for smart contract development, deployment, and performance analysis.
2. **Software:** The research employed a multi-language and multi-platform tool chain. Smart contracts were developed using Solidity and deployed through the Hardhat development environment on the Ethereum Sepolia testnet. JavaScript (via web3.js) was used to script and automate contract interactions. For post-execution analysis, Python 3.10.12 was utilized alongside key libraries such as pandas and matplotlib. Additionally, Ethereum’s JSON-RPC API and public blockchain explorers were used to monitor gas consumption and storage state changes in deployed contracts.
3. **Platform:** The experiments were performed on the Windows 11 operating system.
4. **Development Environment:** While local testing was performed on the device, Google Colab was also used to run Python scripts for data analysis and visualization in a cloud-based, collaborative setting.

7 Limitations

Despite providing valuable insights into Ethereum’s storage-related gas dynamics and their implications for decentralized application (dApp) design, this study is subject to certain limitations:

1. **Testnet Environment Constraints:** The experiments were conducted on the Ethereum Sepolia testnet rather than the mainnet. While Sepolia closely mirrors Ethereum’s behavior, differences in network traffic, miner behavior, and gas pricing volatility may slightly diverge from mainnet conditions.
2. **Exclusion of Layer 2 Protocols:** Though mentioned in the discussion, the practical impact of Layer 2 scaling solutions (e.g., zk-Rollups, Optimism) was

not tested or benchmarked in this study. These solutions could significantly alter the gas economics explored here.

3. **Single DApp Use Case:** The analysis centers around a voting smart contract. Different dApp use cases (NFTs, DeFi, DAOs, etc.) may have varying storage patterns and gas profiles, limiting the generalizability of conclusions.

8 Conclusion and Future Scope

The test points out that Ethereum’s persistent storage is intentionally expensive, influencing the design and optimization of decentralized applications (dApps) by developers. The steep gas cost of adding new data encourages developers to keep on-chain storage as minimal as possible and tends to drive them into seeking off-chain solutions or Layer 2 solutions for data-intensive tasks. In contrast, the partial reimbursement of gas for data deletion encourages storage cleanup responsibility, aligning economic incentives with network efficiency.

These storage dynamics directly impact the scalability and accessibility of Web3 applications. dApp developers need to pay close attention to the trade-offs between on-chain permanence and off-chain efficiency, particularly when handling large or frequently updated datasets. Consequently, most practical applications only store critical data on-chain, employing mechanisms such as hashes or pointers to reference off-chain content, leaving on-chain storage for fundamental state variables that are essential to application integrity.

In summary, Ethereum’s gas fee model and storage cost model are central to the development of the Web 3 ecosystem. While they guarantee network sustainability, they also limit application complexity. Developments in Layer 2 technologies and more efficient storage mechanisms will be crucial to unlocking the full potential of decentralized applications in the emerging Web 3 ecosystem.

Disclosure of Interests. The authors have no competing interests.

References

1. Buterin, V.: On public and private blockchains (2015)
2. Buterin, V., et al.: A next-generation smart contract and decentralized application platform. white paper **3**(37), 2–1 (2014)
3. Guru, D., Perumal, S., Varadarajan, V., Taboada, G.L.: Approaches towards blockchain innovation: A survey and future directions. *Electronics* (2079-9292) **10**(10) (2021)
4. Huang, R., Chen, J., Wang, Y., Bi, T., Nie, L., Zheng, Z.: An overview of web3 technology: Infrastructure, applications, and popularity. *Blockchain: Research and Applications* **5**(1), 100173 (2024)
5. Ji¹, J.Y., Yu¹, W.J., Zhang, J.: 1 sun yat-sen university, guangzhou 510275, china ywj21c@ 163. com 2 south china university of technology, guangzhou 510641, china. In: *Evolutionary Multi-Criterion Optimization: 10th International Conference, EMO 2019, East Lansing, MI, USA, March 10-13, 2019, Proceedings*. vol. 11411, p. 139. Springer (2019)

6. Ou, W., Chen, C., Ren, J.: School of data and computer science, sun yat-sen university, guangzhou, china {ouwj3, chencht3}@ mail2. sysu. edu. cn, issrjt@ mail. sysu. edu. cn. In: Natural Language Processing and Information Systems: 23rd International Conference on Applications of Natural Language to Information Systems, NLDB 2018, Paris, France, June 13-15, 2018, Proceedings. vol. 10859, p. 143. Springer (2018)
7. Panda, S.K., Satapathy, S.C.: An investigation into smart contract deployment on ethereum platform using web3. js and solidity using blockchain. In: Data Engineering and Intelligent Computing: Proceedings of ICICC 2020. pp. 549–561. Springer (2021)
8. Rennock, M.J., Cohn, A., Butcher, J.R.: Blockchain technology and regulatory investigations. *Practical Law Litigation* **1**, 35–44 (2018)
9. Rupa, C., Midhunchakkaravarthy, D., Hasan, M.K., Alhumyani, H., Saeed, R.A.: Industry 5.0: Ethereum blockchain technology based dapp smart contract. *Math. Biosci. Eng* **18**(5), 7010–7027 (2021)
10. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* **151**(2014), 1–32 (2014)
11. Xu, X., Weber, I., Staples, M.: Architecture for blockchain applications (2019)
12. Zamfir, V.: Introducing casper the friendly ghost. *Ethereum Blog* (2015)
13. Zheng, Z., Xie, S., Dai, H., Chen, X., Wang, H.: An overview of blockchain technology: Architecture, consensus, and future trends. In: 2017 IEEE international congress on big data (BigData congress). pp. 557–564. Ieee (2017)